7. Using Decision Statements

Exam Objectives

1. Using Decision Statements
2. Use the decision making statement (if-then and if-then-else)
3. Use the switch statement
4. Compare how == differs between primitives and objects
5. Compare two String objects by using the compareTo and equals methods

7.1 Create if and if/else constructs

7.1.1 Basic syntax of if and if-else

The **if** statement is probably the most used decision construct in Java. It allows you to execute a single statement or a block of statements if a particular condition is true. If that condition is false, the statement (or the block of statements) is not executed. The following are two ways you can write an **if** statement: `if( booleanExpression ) single_statement;`

Notice that there are no curly braces for the statement. The **if** statement ends with the semi-colon. If you have multiple statements that you want to execute instead of just one, you can put all of them within a block like this:

```
if ( booleanExpression ) {
   zero or more statements;
}
```

Observe that there is no semi-colon after the closing curly brace. It is not an error if present though.

If-else statement

The **if-else** statement is similar to an **if** statement except that it also has an **else** part where you can write a statement that you want to execute if the **if** condition evaluates to false:

```
if( booleanExpression ) single statement; //semi-colon required here
else single statement; //semi-colon required here, if-else statement ends with this
    semi-colon
```

Again, if you have multiple statements to execute instead of just one, you can put them within a block:

```
if ( booleanExpression ) single statement; //semi-colon is required here
else {
   zero or more statements;
} //no semi-colon required here, but not an error if it exists.
```

or, if both the **if** and the **else** parts have multiple statements:

```
if ( booleanExpression ) {
   zero or more statements;
} //must NOT have a semi-colon here, error if it exists.
else {
   zero or more statements;
} //no semi-colon required here, but not an error if it exists.
```

If `booleanExpression` evaluates to `true`, the statement (or the block of statements) associated with **if** will be executed and if the `booleanExpression` evaluates to `false`, the **else** part will be executed.

Remember that an empty statement (i.e. just a semicolon) is a valid statement and therefore, the following if and if-else constructs are valid:

```
boolean flag = true;

if(flag); //does nothing, but valid

if(flag); else; //does nothing, but valid

if(flag);else System.out.println(true); //does nothing because flag is true

if(flag) System.out.println(true); else; //prints true
```

By the way, you may see **if/if-else** statements being called **if-then/if-then-else** statement. This is not entirely correct. In some languages such Pascal, "then" is a part of the syntax but it is not in Java. However, "then" is not a keyword in Java and there is no "then" involved in the syntax of **if/if-else**.

7.1.2 Usage of if and if-else in the exam

Let us now look at a few interesting ways if/if-else is used in the exam that might trip you up.

Bad syntax

```
boolean flag = true;
if( flag )
else System.out.println("false"); //compilation error
```

In the above code, there is no statement or a block of statements for the if part. If you don't want to have any code to be executed if the condition is true but want to have code for the else part, you need to put an empty code block for the if part like this:

```
boolean flag = false;
if( flag ) {
}
else System.out.println("false");
```

or

```
boolean flag = false;
if( flag ) ; else System.out.println("false");
```

Instead of having an empty if block, it is better to negate the if condition and put the code in the if block. Like this:

```
boolean flag = false;
if( !flag ) {
  System.out.println("false");
}
```

Bad Indentation

Unlike some languages such as Python, indentation (and extra white spaces, for that matter) holds no special meaning in Java. Indentation is used solely to improve readability of the code. Consider the following two code snippets:

```
boolean flag = false;
if(flag)
System.out.println("false");
else System.out.println("true");
{
System.out.println("false");
}
```

and

```
boolean flag = false;
if(flag)
  System.out.println("false");
else
  System.out.println("true");

{
  System.out.println("false");
}
```

The above two code snippets are equivalent. However, since the second snippet is properly indented, it is easy to understand that the last code block is not really associated with the if/else statement. It is an independent block of code and will be executed irrespective of the value of `flag`. This code will, therefore, print `true` and `false`.

Here is another example of bad indentation:

```
boolean flag = false;
int i = 0;
if(flag)
   i = i +1;
   System.out.println("true");
else
   i = i + 2;
   System.out.println("false");
```

The above code is trying to confuse you into thinking that there are two statements in the if part and two statements in the else part. But, with proper indentation, it is clear what this code is really up to:

```
boolean flag = false;
int i = 0;

if(flag)  i = i +1;
```

```
System.out.println("true");

else i = i + 2;

System.out.println("false");
```

It turns out that the last three lines of code are independent statements. The else statement is completely out of place because it is not associated with the if statement at all and will, therefore, cause compilation error.

Missing else

As we saw earlier, the else part is not mandatory in an if statement. You can have just the if statement. But when coupled with bad indentation, an if statement may become hard to understand as shown in the following code:

```
boolean flag = true;
if(flag)
System.out.println("true");
{
System.out.println("false");
}
```

The above code prints `true` and `false` because there is no **else** part in this code. The second `println` statement is in an independent block and is not a part of the **if** statement.

Dangling else

"Dangling else" is a well known problem in programming languages that have **if** as well as **if-else** statements. This is illustrated in the following piece of code that has two **if** parts but only one **else** part:

```
int value = 3;
if(value == 0)
if(value == 1) System.out.println("b");
else System.out.println("c");
```

The question here is with which **if** should the **else** be associated? There are two equally reasonable answers to this question as shown below:

```
int value = 3;
if(value == 0) {
    if(value == 1) System.out.println("b");
}
else System.out.println("c");
```

and

```
int value = 3;
if(value == 0) {
   if(value == 1)
      System.out.println("b");
   else
       System.out.println("c");
}
```

In the first interpretation, **else** is associated with the first **if**, while in the second interpretation, **else** is associated with the second **if**. If we go by the first interpretation, the code will print c, and if we go by the second interpretation, the code will not print anything. For a compiler, both are legally valid ways to interpret the code, which makes the code ambiguous.

Since neither of the interpretations is more correct than the other, Java language designers broke the tie by deciding to go with the second interpretation, i.e., the **else** is to be associated with the nearest **if**. That is why the above code does not print anything as there is no **else** part associated with the first **if**. Based on the above discussion, you should now be able to tell the output of the following code:

```
int value = 3;
   if(value == 0)
   if(value == 1)
      System.out.println("b");
   else
       System.out.println("c");
   else
       System.out.println("d");
```

Just follow the rule that an else has to be associated with the nearest if. The following is how the above statement will be grouped:

```
int value = 3;

   if(value == 0){
       if(value == 1)
           System.out.println("b");
       else
           System.out.println("c");
   }
   else System.out.println("d");
```

It will print d.

Using assignment statement in the if condition

Recall that every assignment statement itself is a valid expression with a value of its own. Its type and value are the same as the ones of the target variable. This fact can be used to write a very tricky if statement as shown below:

```
boolean flag = false;
if(flag = true){
  System.out.println("true");
}
else {
 System.out.println("false");
}
```

Observe that `flag` is not being compared with `true` here. It is being assigned the value `true`. Thus, the value of the expression `flag = true` is true and that is why the **if** part of the statement is executed instead of the **else** part. While this type of code is not appreciated in a professional environment, a similar construct is quite common:

```
String data = null;
if( (data = readData()) != null ) //assuming that readData() returns a String
{
  //do something
}
```

The above code is fine because the assignment operation is clearly separated from the comparison operation. The value of the expression `data = readData()` is being compared with `null`. Remember that the value of this expression is the same as the value that is being assigned to `data`. Thus, the `if` body will be entered only if `data` is assigned a non-null value.

Using pre and post increment operations in the if condition

You will see conditions that use pre and post increment (and decrement) operators in the exam. Something like this:

```
int x = 0;
if(x++ > 0){ //line 2
  x--; //line 3
}

if (++x == 2){ //line 6
  x++; //line 7
}
System.out.println(x);
```

You can spot the trick easily if you have understood the difference between the value of an expression and the value of a variable used in that expression (I explained this in the "Using Operators" chapter). At line 2, `x` will be incremented to 1 but value of the expression `x++` is 0 and therefore, the condition will be evaluated to `false`. Thus, line 3 will not be executed. At line 6, `x` is incremented to 2 and the value of the expression `++x` is also 2. Therefore, the condition will be evaluated to `true` and line 7 will be executed, thereby increasing the value of `x` to 3. Thus, the above code will print 3.

Remember that conditions are used in ternary expressions and loops as well, so, you need to watch out for this trick there also.

7.2 Create ternary constructs

7.2.1 The ternary conditional operator ?:

> The ternary operator (?:) is not mentioned in the JFCJA exam objectives explicitly. We haven't seen any one getting a question on it either. However, it is one of the commonly used operators in Java and from that perspective, you should know about it. You may skip through the following discussion if you are short on time.

The syntax of the ternary operator is as follows:

```
operand 1 ?  operand 2 :  operand 3;
```

Operand 1 must be an expression that returns a `boolean`. The `boolean` value of this expression is used to decide which one of the rest of the two other operands should be evaluated. In other words, which of the operands 2 and 3 will be evaluated is conditioned upon the return value of operand 1. If the boolean expression given in operand 1 returns true, the ternary operator evaluates and returns the value of operand 2 and if it is false, it evaluates and returns the value of operand 3. From this perspective, it is also a **"conditional operator"** (as opposed to `&`, `|`, `!`, and `^`, which are really just **"logical"** operators).

Examples:

```
boolean sweet = false;
int calories = sweet ? 200 : 100; //assigns 100 to calories
boolean sweetflag = (calories == 100 ? false : true); //assigns false to sweetflag

boolean hardcoded = false;
//assuming getRateFromDB() method returns a double
double rate = hardcoded ? 10.0 : getRateFromDB(); //invokes method getRateFromDB

String value = sweetflag ? "Sweetened" : "Unsweetened";

Object obj = sweetflag ? "Sweetened" : new Object();
```

The ternary conditional operator is often thought of a short form for the **if/else statement** but it is similar to the if/else statement only up to conditional evaluation of its other two operands part. Their fundamental difference lies in the fact that the ternary expression is just an **expression** while an if/else statement is a **statement**. As discussed earlier, every expression must have a value and so, must the ternary expression. Since the value of a ternary expression is the value returned by the second or the third operand, the second and third operands of the ternary operator can comprise any expression. As the example given above shows, they can also include **non-void method invocations**. There is no such restriction with the if/else statement. The following example highlights this difference:

```
boolean flag = true;
if(flag) System.out.println("true");
else System.out.println("false");
```

The above if/else statement compiles fine and prints `true` but a similar code with ternary expression does not compile.

```
flag ? System.out.println("true") : System.out.println("false");
```

The reason for non-compilation of the above code is two fold. The first is that a ternary expression is not a valid statement on its own, which means, you cannot just have a free standing ternary expression. It can only be a part of a valid statement such as an assignment. For example,

```
int x = flag ?  System.out.println("true")
:  System.out.println("false");
```

This brings us to the second reason. The second and third operands in this example are invocations methods that return `void`. Obviously, you cannot assign void to an int variable. In fact, you cannot assign void to any kind of variable. Therefore, it fails to compile.

Type of a ternary conditional expression

Now that we have established that a ternary conditional expression must return a value, all that is left to discuss is the type of the value that it can return. It can return values of three types: **boolean, numeric,** and **reference**. (There are no other types left for that matter!)

If the second and third operands are expressions of type boolean (or Boolean), then the return type of the ternary expression is boolean. For example:

```
int a = 1, b = 2;
boolean flag = a == b? true : false; //ternary expression that returns a boolean
```

If the second and third operands are expressions of numeric types (or their wrapper classes), then the return type of the ternary expression is the wider of the two numeric types. For example, `double d = a == b?  5 :  10.0;`. Observe here that the second operand is of type `int` while the third is of type `double`. Since `double` is wider than `int`, the type of this ternary expression is `double`. You cannot, therefore, do **`int`** `d = a == b?  5 :  10.0;` because you cannot assign a double value to an int variable without a cast.

If the second and third operands are neither of the above, then the return type of the ternary expression is reference. For example,
`Object str = a == b?  "good bye" :  "hello";` Here operands 2 and 3 are neither numeric nor boolean. Therefore, the return type of this ternary expression is a reference type.

The first two types are straight forward but the third type begs a little more discussion. Consider the following line of code:

`Object obj = a == b?  5 :  "hello";.`

Here, the second operand is of a numeric type while the third is of type `String`. Since this falls in the third category, the return type of the expression `a == b? 5 : "hello";` must be a reference. The question before the compiler now is what should be the type of the reference that is returned by the expression. One operand is an `int`, which can be boxed into an `Integer` object and another one is a `String` object. If the `boolean` condition evaluates to `true`, the expression will return an `Integer` object and if the condition evaluates to `false`, the expression will return a `String` object. Remember that the compiler cannot execute any code and so, it cannot determine what the expression will return at run time. Thus, it needs to pick a type that is compatible with both kind of values. The compiler solves this problem by deciding to pick the most specific common superclass of the two types as the type of the expression. In this case, that class is `java.lang.Object`. By selecting the most specific common super class, the compiler ensures that irrespective of the result of the condition, the value returned by this expression will always be of the same type, i.e., `java.lang.Object`, in this case. Here is an example, where the most specific common super class is not `Object` :

```
Double d = 10.0;
Byte by = 1;
Number n = a == b? d : by;
```

Here, the most specific common superclass of `Double` and `Byte` is `Number`. (Recall that all wrapper classes for integral types extend from `java.lang.Number`, which in turn extends from `java.lang.Object`). You can therefore, assign the value of the expression to a variable of type `Number`.

You should now be able to tell the result of the following two lines of code:

```
int value = a ==  b? 10 : "hello"; //1
```

```
System.out.println(a ==  b? 10 : "hello"); //2
```

As discussed above, the type of the expression `a == b? 10 : "hello";` is `Object`. Can you assign an `Object` to an `int` variable? No, right? Therefore, the first line will not compile. Can you pass an `Object` to the `println` method? Of course, you can. Therefore, the second line will compile and run fine.

The short circuiting property of ?:

Depending on whether the value of operand one is true or false, either operand 2 or operand 3 is evaluated. In other words, if the condition is `true`, operand 3 is not evaluated and if the condition is `false`, operand 2 is not evaluated. In this respect, the ternary conditional operator is similar to the other two short circuiting operators, i.e., `&&` and `||`. However, there is an important difference between the two. While with `&&` and `||`, evaluation of both the operands is possible in certain situations, it is never the case with `?:` operator. `?:` evaluates exactly one of the two operands in all situations.

The short circuiting nature of `?:` provides a good opportunity for trick questions in the exam. For example, what will the following code print?

```
int x = 0;
int y = 1;
System.out.println(x>y? ++x : y++);
System.out.println(x+" "+y);
```

This code prints:

```
1
0 2
```

Since the value of x is not greater than y, x> y evaluates to `false` and therefore, the ternary expression returns the value of the third operand, which is y++. Since y++ uses post-increment operator, the return value of y++ will be the current value of y, which is 1. y will then be incremented to 2. Observe that evaluation of the second operand is short circuited because the first operand evaluates to false. Therefore, ++x is never executed. Thus, the second print statement prints 0 2.

7.3 Use a switch statement

7.3.1 Creating a switch statement

A switch statement allows you to use the value of a variable to select which code block (or blocks) out of multiple code blocks should be executed. Here is an example:

```
public class TestClass {
   public static void main(String[] args){
      int i = args.length;

      switch(i) {  //switch block starts here

          case 0 : System.out.println("No argument");
                   break;
          case 1 : System.out.println("Only one argument");
                   break;
          case 2 : System.out.println("Two arguments");
                   break;
          default : System.out.println("Too many arguments!");
                   break;

       } //switch block ends here

       System.out.println("All done.");
   }
}
```

There are four blocks of code in the above switch statement. Each block of code is associated with a **case label**. Depending of the value of the variable i, the control will enter the code block associated with that particular case label and keep on executing statements until it finds a `break`

statement. For example, if the value of `i` is `0`, the control will enter the first code block. It will print `No argument` and then encounter the `break` statement. The break statement causes the control to exit the switch statement and move on to the next statement after the switch block, which prints `"All done"`.

If the value of `i` doesn't match with any of the case labels, the control looks for a block labelled **default** and enters that block. If there is no default block either, the control does not enter the switch block at all. Since this "switching" is done based on the expression specified in the switch statement (which is just `i` in this example), this expression is aptly called the "**switch expression**".

Operationally, this seems quite similar to a cascaded **if/else** statement. Indeed, the above code can very well be written using an if/else statement as follows:

```
public class TestClass {
   public static void main(String[] args){
      int i = args.length;

      if(i == 0) {
          System.out.println("No argument");
       } else if (i == 1) {
          System.out.println("Only one argument");
       } else if (i == 2) {
          System.out.println("Two arguments");
       } else {
          System.out.println("Too many arguments!");
       }
       System.out.println("All done.");
   }
}
```

Well, why do you need a switch statement then, you may ask. To begin with, as you can see, an if/else statement is a lot more verbose than a switch statement. The switch version is also a little easier to comprehend than the if/else version. But underneath this syntactic ease lies a complicated beast. This is evident when we look at the moving parts involved in a switch statement more closely.

The switch expression

A switch expression must evaluate to one of the following three kinds:

1. a limited set of integral types (`byte`, `char`, `short`, `int`), and their wrapper classes. Observe that even though `long` is an integral type, it cannot be the type of a switch variable. `boolean`, `float`, and `double` are not integral types anyway and therefore, cannot be the type of a switch variable either.

2. **enum** type

3. `java.lang.String` - Generally, reference types are not allowed as switch expressions but an exception for java.lang.String was made in Java 7. So now, you can use a String expression as a switch expression.

Compare this to an **if/else** statement where branching is done based on the value of a **boolean expression**. This limits an if/else statement to at most two branches. Of course, as we saw earlier, you can cascade multiple if/else statements to achieve multiple branches.

The case labels

Case labels must consist of **compile time constants** that are assignable to the type of the switch expression. For example, if your switch expression is of type `byte`, you cannot have a case label with a value that is larger than a byte. Similarly, if your switch expression is of type `String`, the case labels must be constant string values as illustrated in the following code:

```
public static void main(String[] args){
 switch(args[0]){
   case "1" : System.out.println("one"); //valid because "1" is a compile time constant
   case "1"+"2" : System.out.println("one"); //valid because "1"+"2" is a compile time
    constant
   case args[1] : System.out.println("same args"); //will not compile because args[1]
    is not a compile time constant
   case "abc".toUpperCase() : System.out.println("ABC"); //will not compile because
    "abc".toUpperCase() is not a compile time constant
 }
}
```

Observe that `"1"` and `"1"+"2"` are compile time constants because the value of these expressions is known at compile time, while `args[1]` and `"abc".toUpperCase()` are not compile time constants because their values can only be determined at run time when the code is executed.

The interesting thing about case labels is that they are **optional**. In other words, a switch statement doesn't necessarily have to have a case label. The following is, therefore, a superfluous yet valid switch statement.

```
switch(i){
  default :  System.out.println("This will always be printed");
}
```

Another point worth repeating here is that although it is very common to use a single variable as the switch expression but you can use any expression inside the switch. And when you talk of an expression, all the baggage of numeric promotion, casting, and operator precedence that we saw previously, comes along with it. You need to consider all that while checking the validity of case labels. For example, while the following code fails to compile:

```
byte b = 10;
switch(b){ //type of the switch expression here is byte
   case 1000 : //1000 is too large to fit into a byte
         System.out.println("hello!");
}
```

the following code compiles fine:

```
byte b = 10;
switch(b+1){ //type of the switch expression here is now int due to numeric promotion
```

```
   case 1000 : //1000 can fit into an int
         System.out.println("hello!");
}
```

The default block

There can be at most one default block in a switch statement. The purpose of the default block is to specify a block of code that needs to be executed if the value of the switch expression does not match with any of the case labels. Just like the case labels, this block is also **optional**.

The order of case and default blocks

Java does not impose any particular order for the case statements and the default block. Thus, although it is customary to have the default block at the end of a switch block, you can have it even at the beginning. Similarly, Java does not care about the ordering of the case labels.

However, "does not care" does not mean "not important"! Ordering of case and default blocks becomes very important in combination with the use of the `break` statement as we will see next.

The break statement

I mentioned in the beginning that the case labels determine the entry point into a switch statement and the break statement determines the exit. That is true but the interesting thing is that even the break statement is **optional**. A case block does not necessarily have to end with a break. Let me modify the program that I showed you in the beginning:

```
public class TestClass {
   public static void main(String[] args){

      switch(args.length) {  //switch block starts here

          case 0 : System.out.println("No argument");
                   //break;
          case 1 : System.out.println("Only one argument");
                   //break;
          case 2 : System.out.println("Two arguments");
                   //break;
          default : System.out.println("Too many arguments!");
                   //break;

       } //switch block ends here

       System.out.println("All done.");
   }
}
```

I have commented out all the break statements. Now, if you run this program **without any argument**, you will see the following output:

```
No argument
Only one argument
Two arguments
Too many arguments!
All done.
```

The control entered at the block labelled **case 0** (because `args.length` is 0), and executed all the statements of the switch block...even the statements associated with other case blocks that did not match the value of `args.length`. This is called **"fall through"** behavior of a switch statement. In absence of a break statement, the control falls through to the next case block and the next case block, and so on until it reaches the end of the switch statement. This feature is used when you want to have one code block to execute for multiple values of the switch expression. Here is an example:

```
char ch = 0;
int noOfVowels = 0;
while( (ch = readCharFromStream()) > 0) {
  switch(ch) {

   case 'a' :
   case 'e' :
   case 'i' :
   case 'o' :
   case 'u' :
          noOfVowels++;

   default :   logCharToWhatever(ch);
   }
}
System.out.println("Number of vowels received "+noOfVowels);
```

The above code logs each character it receives but increments `noOfVowels` only if the character received is a lower case vowel.

You will see questions in the exam on this behavior of the switch statement. So pay close attention to where in the switch block does the control enter and where it exits. Always check for missing break statements that cause the control to fall through to the next case block.

The following is a typical code snippet you may get in an exam. Try running it with different arguments and observe the output in each case:

```
public class TestClass {
   public static void main(String[] args){
      int i = 0;
      switch(args[0]) {
```

```
            default : i = i + 3;
            case "2" : i = i + 2;
            case "0" : break;
            case "1" : i = i + 1;


        }

        System.out.println("i is "+i);
    }
}
```

The fall through behavior of a switch statement does not really get used a lot in Java but it has been used in interesting ways to optimize code in other languages. If you have time, you might want to google "duff's device" to see one such usage. This is, of course, not required for the exam :)

7.4 Exercise

1. Write a method that accepts a number as input and prints whether the number is odd or even using an if/else statement as well as a ternary expression.

2. Accept a number between 0 to 5 as input and print the sum of numbers from 1 to the input number using code that exploits the "fall through" behavior of a switch statement.

3. Accept a number as input and generate output as follows using a cascaded and/or nested if/else statement - if the number is even print "even", if it is divisible by 3, print "three", if it is divisible by 5, print "five" and if it is not divisible by 2, 3, or 5, print "unknown". If the number is divisible by 2 as well as by 3, print "23", and if the number is divisible by 2, 3, and 5, print "235".

4. Indent the following code manually such that it reflects correct if - else associations. Use a plain text editor such as Notepad. Copy the code into a Java editor such Netbeans or Eclipse and format it using the editor's auto code formatting function. Compare your formatting with the editor's.

```
int a = 0, b = 0, c = 0, d = 0;
boolean flag = false;
if (a == b)
if (c == 10)
{
if (d > a)
{
} else {
```

```
}
if (flag)
System.out.println("");
else
System.out.println("");
}
else if (flag == false)
System.out.println("");
else if (a + b < d) {
System.out.println("");
}
else
System.out.println("");
else d = b;
```

8. Using Looping Statements

Exam Objectives

1. Using Looping Statements
2. Describe looping statements
3. Use a for loop including an enhanced for loop
4. Use a while loop
5. Use a do- while loop
6. Compare and contrast the for, while, and do-while loops
7. Develop code that uses break and continue statements

8.1 What is a loop

8.1.1 What is a loop

A loop causes a set of instructions to execute repeatedly until a certain condition is reached. It's like when the kids keep asking, "are we there yet?", when you are on a long drive in a car. They ask this question in a "loop", until you are really there :) Or until you "break" their loop by putting on their favorite music video.

Well, in Java, loops work similarly. They let you execute a group of statements multiple times or even forever depending on the **loop condition** or until you **break** out of them. Every repetition of execution of the statements is called an **iteration**. So, for example, if a group of statements is executed 10 times, we can say that the loop ran for 10 iterations.

Loops are a fundamental building block of **structured programming** and are used extensively for tasks ranging from the simple such as counting the sum of a given set of numbers to the complicated such as showing a dialog box to the user until they select a valid file.

Java has three different ways in which you can create a loop - using a while statement, using a do/while statement, and using a for statement. Let us take a look at each of them one by one.

8.2 Create and use while loops

8.2.1 The while loop

As the name of this loop suggests, a while loop executes a group of statements **while** a condition is true. In other words, it checks a condition and if that condition is true, it executes the given group of statements. After execution of the statements, i.e., after finishing that iteration, it loops back to check the condition again. If the condition is false, the group of statements is not executed and the control goes to the next statement after the while block. The syntax of a while loop is as follows:

```
while (boolean_expression) {
     statement(s);
}
```

and here is an example of its usage:

```
public class TestClass {
   public static void main(String[] args){
      int i = 4;
      while(i>0){
        i--;
        System.out.println("i is "+i);
      }
     System.out.println("Value of i after the loop is "+i);
   }
}
```

It produces the following output:

```
i is 3
i is 2
i is 1
i is 0
Value of i after the loop is 0
```

Observe that the condition is checked **before** the group of statements is executed and that once the condition `i> 0` returns `false`, the control goes to the next statement after the while block.

As you have seen in the past with if/else blocks, if you have only a single statement that you need to execute in a loop, you may get rid of the curly braces if you want. In this case, the syntax becomes:

```
while(boolean_expression) statement;
```

The example program given above can also be written to use a single statement like this:

```
public class TestClass {
   public static void main(String[] args){
      int i = 4;
      while(i-->0) System.out.println("i is "+i); //no curly braces
      System.out.println("Value of i after the loop is "+i);
   }
}
```

The above code looks the same as the previous one but produces a slightly different output:

```
i is 3
i is 2
i is 1
i is 0
Value of i after the loop is -1
```

The difference is that I have used the post-decrement operator to decrement the value of `i` within the condition expression itself. Remember that when you use the **post**-decrement (or the **post**-increment) operator, the variable is decremented (or incremented) **after** its existing value is used to evaluate the expression. In this case, when `i` is `0`, the condition expression evaluates to false and the while block is not executed, but the variable `i` is nevertheless decremented to `-1`. Therefore, the value of `i` after the loop is `-1`.

8.2.2 Using a while loop

Control condition and control variable

When using a while loop you should be careful about the expression that you use as the while condition. Most of the time, a while condition comprises a single variable (which is also called the **"control variable"**, because it controls whether the loop will be entered or not) that you compare against a value. For example, `i> 4` or `name != null` or `bytesRead != -1` and so on. However, it can get arbitrarily large and complex. For example, `account == null || (account != null &&`

`account.accountId == null) || (account != null && account.balance == 0)`. The expression must, however, return a `boolean`. Unlike some languages such as C, Java does not allow you to use an integer value as the while condition. Thus, `while(1) System.out.println("loop forever");` will not compile.

while body

Ideally, the code in the while body should modify the control variable in such a way that the control condition will evaluate to false when it is time to end the loop. For example, if you are processing an array of integers, your control variable could be set to the index of the element that you are processing, and each iteration should increment that variable. For example,

```
int[] myArrayOfInts = //code to fetch the data
int i = 0; //control variable
while(i<myArrayOfInts.length){
//do something with myArrayOfInts[i]
i++; //increment i so that the control condition will evaluate to false after the last
    element is processed
}
```

There are situations where you do not want a loop to end at all. For example, a program that listens on a socket for connections from clients may use a while loop as follows:

```
Socket clientSocket = null;
while( (clientSocket = serverSocket.accept() ) != null ){
  //code to hand over the clientSocket to another thread and go back to the while
    condition to keep listening for connection requests from clients
}
```

The above is a commonly used while loop idiom.

A never ending while loop can also be as simple as this:

```
while(true){
  System.out.println("keep printing this line forever!");
}
```

On the other extreme, keep an eye for a condition that never lets the control enter the while body:

```
int i = 0;
while(i>0){ // the condition is false to begin with
  System.out.println("hello"); //this will never be printed
   i++;
}
```

It is possible to exit out of a while loop without checking the while condition. This is done using the `break` statement. I will discuss it later.

8.3 Create and use do/while loops

8.3.1 The do-while loop

A do-while loop is similar to the while loop. The only difference between the two is that in a do-while loop the loop condition is checked after executing the loop body. Its syntax is as follows:

```
do {
  statement(s);
}while(boolean_expression);
```

Observe that a do-while statement starts with a "do" but there is no "do" anywhere in a while statement. Another important point to understand here is that since the loop condition is checked after the loop body is executed, the loop body will always be executed at least once.

As with a while statement, it is ok to remove the curly braces for the loop body if there is only one statement in the body. Thus, the following two code snippets are actually the same:

```
int i = 4;
do {
   System.out.println("i is "+i);
} while(i-->0);
System.out.println("Loop finished. i is "+i);


int i = 4;
do
   System.out.println("i is "+i);
while(i-->0);
System.out.println("Loop finished. i is "+i);
```

Deciding whether to use a while loop or a do while loop is easy. If you know that the loop condition may evaluate to false at the beginning itself, i.e., if the loop body may not execute even once, you must use a while loop because it lets you check the condition first and then execute the body. For example, consider the following code:

```
Iterator<Account> acctIterator = accounts.iterator();
while(acctIterator.hasNext()){  //no need to enter the loop body if accounts collection
    is empty
   Account acct =  acctIterator.next();
   //do something with acct
}
```

Don't worry about the usage of `Iterator` or `<Account>` in the above code. The point to understand here is that you want to process each account object in the accounts collection and if there is no account object, you don't want to enter the loop body at all. If you use a do-while loop, the code will look like this:

```
Iterator<Account> acctIterator = accounts.iterator();
do {
```

```
   Account acct =  acctIterator.next(); //will throw an exception
   //do something with acct
}while(acctIterator.hasNext());
```

The above code will work fine in most cases but will throw an exception if the account collection is empty. To achieve the same result with a do-while loop, you would have to write an additional check for an empty collection at the beginning. Something like this:

```
Iterator<Account> acctIterator = accounts.iterator();
if(acctIterator.hasNext()) { //no need to enter the if body if accounts collection is
    empty
   do{
       Account acct =  acctIterator.next();
       //do something with acct
   }while(acctIterator.hasNext())
}
```

I think the choice is quite clear. A while loop is a natural fit in this case.

Generally, a while loop is considered more readable than a do-while loop and is also used a lot more in practice. I have not needed to use a do-while loop in a long while myself :)

8.4 Create and use for loops

8.4.1 Going from a while loop to a for loop

The **for loop** is the big daddy of loops. It is the most flexible, the most complicated, and the most used of all loop statements. It has so many different flavors that many programmers do not get to use some of its variations despite years of programming in Java. But don't be scared. It still follows the basic idea of a loop, which is to let you execute a group of statements multiple times.

To ease you into it, I will morph the code for a while loop into a for loop. Here is the code that uses a while loop:

```
public class TestClass {
  public static void main(String[] args){
    int i = 4;
    while(i>0){
      System.out.println("i is "+i);
      i--;
    }
    System.out.println("Value of i after the loop is "+i);
  }
}
```

The output of the above code is:

```
i is 4
i is 3
```

```
i is 2
i is 1
Value of i after the loop is 0
```

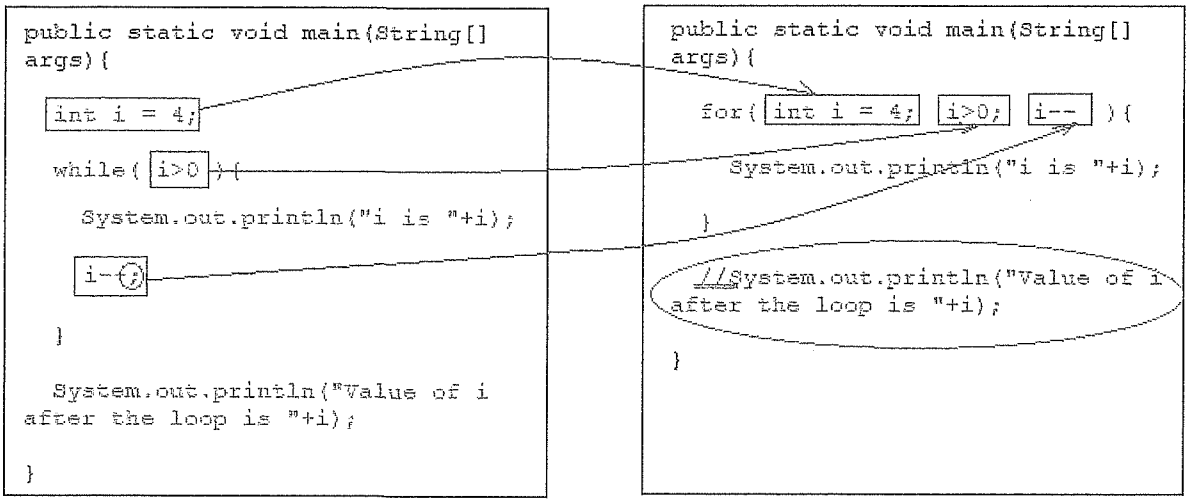
Here is the same code with a for loop:

```
public class TestClass {
  public static void main(String[] args){
    for(int i = 4; i>0; i--){
      System.out.println("i is "+i);
    }
    //System.out.println("Value of i after the loop is "+i);
  }
}
```

Its output is:

```
i is 4
i is 3
i is 2
i is 1
```

As you can see, it produces the same output except the last line. The following image shows how the elements of a while loop of the first code were mapped to the for loop of the second code.



Converting a while loop to a for loop

Here are a few things that you should observe in the transformation shown above.

1. The while statement contains just the comparison but the for statement contains initialization, comparison, and updation. Technically, the declaration of the loop variable i and its updation through `i--` is not really a part of the while statement. But the for loop has a provision for both.

2. The implication of declaring the loop variable `i` in a for statement is that it is scoped only to the for block and is not visible outside the for block, which is why I have commented out the last print statement in the second code.

3. There is no semi-colon after `i--` in the for statement.

This example illustrates that fundamentally there is not much difference between the two loop constructs. Both the loops have the same three basic components - declaration and/or initialization of a loop variable, a boolean expression that determines whether to continue next iteration of the loop, and a statement that updates the loop variable. But you can see that the for loop has a compact syntax and an in-built mechanism to control the initialization and updation of the loop variable besides the comparison as compared to the while loop.

8.4.2 Syntax of a for loop

A for loop has the following syntax:

```
for( optional initialization section ; optional condition section ; optional updation
    section ) {
  statement(s);
}
```

Predictably, if you have only zero or one statement in the for block, you can get rid of the curly braces and end the statement with a semicolon like this:

```
for( optional initialization section ; optional condition section ; optional updation
    section ) ;
//no body at all
```

or

```
for( optional initialization section ; optional condition section ; optional updation
    section )
     single_statement;
```

All three sections of a for statement are optional but the **two semi-colons** that separate the three sections are not. You are allowed to omit one, two, or all of the three sections. Thus, `for( ; ; );` is actually a valid for statement but `for();` and `for(;);`are not.

Order of execution

Various pieces of a for loop are executed in a specific order, which is as follows:

1. The **initialization section** is the first that is executed. It is executed **exactly once** irrespective of how many times the for loop iterates. If this section is **empty**, it is **ignored**.

2. Next, the **condition section** is executed. If the result of the expression in this section is true, the next iteration, i.e., execution of the statements in the for block is kicked off. If the result is false, the loop is terminated immediately, i.e., neither the for block and nor the updation section is executed. If this section is **empty**, it is assumed to be **true**.

3. The statements in the **for block** are executed.

4. The **updation section** is executed. If this section is **empty**, it is **ignored**.

5. The control goes back to the **condition section**.

Let us see how the above steps are performed in the context of the following for loop.

```
for(int i = 3; i>0; i--){
    System.out.println("i is "+i);
  }
```

1. `int i = 3;` is executed. So, `i` is now 3.

2. Expression `i> 0` is evaluated and since 3 indeed greater than 0, it evaluates to `true`. Therefore, the next iteration will now commence.

3. The print statement is executed, which prints 3.

4. `i--` is executed thereby reducing `i` to 2.

5. Control goes back to the condition section. `i> 0` evaluates to `true` because 2 is greater than 0. Therefore, the next iteration will now commence.

6. The print statement is executed, which prints 2.

7. `i--` is executed thereby reducing `i` to 1.

8. Control goes back to the condition section. `i> 0` evaluates to `true` because 1 is greater than 0. Therefore, the next iteration will now commence.

9. The print statement is executed, which prints 1.

10. `i--` is executed thereby reducing `i` to 0.

11. Control goes back to the condition section. `i> 0` evaluates to `false` because 0 is not greater than 0. Therefore, the loop will be terminated. The statements in the for block and the updation section will not be executed and the control goes to the next statement after the for block (which is the end of the method in this case).

So far, the for statement looks quite straight forward. You will see the complications when we turn our attention to the intricacies of the three sections of the for statement next.

8.4.3 Parts of a for loop

The initialization section

The initialization section allows you to specify code that will be executed at the beginning of the for loop. This code must belong to a category of statements that are called **"expression statements"**. Expression statements are expressions that can be used as statements on their own. These are: **assignment**, **pre/post increment/decrement expression**, **a method call**, and **a class instance creation expression** (e.g. using the new operator). Besides expression statements, this section also allows you to declare local variables.

Here are a few examples of valid expression statements in a for loop:

```
int i = 0;
for(i = 5; i<10; i++); //assignment

Object obj;
for(obj = "hello"; i<10; i++); //assignment

int i = 0;
int k = 0;
Object obj = "";
for(i = 0, k = 7, obj = "hello";i<10; i++); //multiple assignments

int k = 0;
for(++k; i<10; i++); //pre-increment

for(new ArrayList(); i<10; i++); //instance creation

int i = 0;
for(System.out.println("starting the loop now"); i<10; i++); //method call

for(k++, i--, new String();i<10; i++); //multiple expressions
```

Observe that there doesn't need to be any relationship between the variable used in the initialization section and the variable used in other sections of a for loop.

The following are a few examples of valid local variable declarations:

```
for(int i = 5; i<10; i++); //single variable declaration

for(int i = 5, k = 7; i<10; i++); //multiple variable declaration
```

You can only declare variables of one type. So the following is **invalid** :

```
for(int i = 5,  String str  = ""; i<10; i++); //invalid
```

Redeclaring a variable is also invalid:

```
int i = 0;
for(int i = 5; i<10; i++); //invalid because i is already declared
```

Another important point to note here is that a variable declared in the initialization section is visible only in the for loop. Thus, the following **will not compile** because i is not visible outside the loop.

```
for(int i = 5; i<10; i++){
  System.out.println("i is "+i);
}
System.out.println("Final value of i is "+i); //this line will not compile
```

The condition section

This one is simple. No, really :) You can only have an expression that returns a `boolean` or `Boolean` in this section. If there is no expression in this section, the condition is assumed to be `true`.

The updation section

The rules for the updation section are the same as the initialization section except that you cannot have any declarations here. This section allows only "expression statements", which I have already discussed above. Generally, this section is used to update the loop variable (`i++` or `k = k + 2` and so on) but as you saw in examples above, this is not the only way to use it. There doesn't need to be any relationship between the code specified here and in other sections. The following are a few valid examples:

```
int i = 0;
for(;i<10; i++); //post-increment

for(;i<10; i = i + 2); //increment by two

for(;i<10; i = someRef.getValue()) //assignment

for(Object obj = new Object(); obj != null; ) { //empty updation section
   System.out.println(obj);
   obj = null;
}

for(int i = 0; i<10; callSomeMethod() ); //method call
```

Observe that there is no semi-colon after the expression statement. It is terminated by the closing parenthesis of the for statement.

An infinite for loop

Based on the above information, it should now be very easy to analyze the following loop:

```
for( ; ; ) ;
```

The initialization section is empty. The condition section is empty, which means it will assumed to be `true`. The updation section is empty. The loop code is also empty. There is nothing that

makes the condition section to evaluate to `false` and therefore, there is nothing to stop this loop from iterating forever.

Now, as an exercise, try analyzing the following loop and determine its output:

```
boolean b = false;
for(int i=0 ; b = !b ; ) {
  System.out.println(i++);
}
```

8.5 Create and use for each loops

8.5.1 The enhanced for loop

Motive behind the enhanced for loop

In professional Java programming, looping through a collection of objects is a very common requirement. Here is how one might do it:

```
String[] values = { "a", "bb", "ccc" };
for(int i = 0; i<values.length; i++){
  System.out.println(values[i]); //do something with each value
}
```

The above code iterates through an array but the same pattern can be used to iterate through any other collection such as a List. You can code this loop using a while or do-while construct as well.

Java designers thought that this is too much code to write for performing such a simple task. The creation of an iteration variable `i` and the code in the condition section to check for collection boundary is necessary only because of the way the for (or other) loops work. They have no purpose from a business logic point of view. All you want is a way to do something with each object of a collection. You don't really care about the index at which that element is inside that collection.

Furthermore, some collections such as a set have no notion of index. Although you are not expected to know about sets for the JFCJA exam, you can't really avoid knowing a bit about it if you want to understand the "enhanced for loop". So, briefly, `java.util.HashSet` is a class in standard Java library that allows you to collect a bunch of unique objects but without any specific order. Unlike an array, where all the elements are ordered (note that I am saying ordered, not sorted) and each element is accessible through its index, there is no index in a `HashSet`. You can't tell which element is the first element and which one is the last because there is no order. How will you iterate through every element of a `HashSet` then?

Here is the code that iterates through the elements of a `HashSet` just to give you an idea of how you may iterate though a collection of objects that does not support index based retrieval:

```
import java.util.HashSet;
import java.util.Iterator;
public class TestClass{
   public static void main(String[] args){
      Set s = new HashSet();
      s.add("a");
      s.add("bb");
      s.add("ccc");

      Iterator it = s.iterator();

      while( it.hasNext() ){
         Object value = it.next();
         System.out.println(value); //do something with each value
      }
   }
}
```

Don't worry if you feel overwhelmed by the above code. The only purpose of the above code is to show you how cumbersome it was to iterate through the objects of a collection. The above code creates an `Iterator` object, which has no relation to the business logic, to iterate through the elements. Such code that is not required by the business logic but is required only because of the way a programming language works is called "boilerplate code". Such code can be easily eliminated if a programming language provides higher level constructs that internalize the logic of this code.

In Java 5, Java designers simplified the iteration process by doing exactly that. They introduced a new construct that internalizes the creation of iteration variable and the boundary check and called it the "enhanced for loop"or the "for-each loop".

The enhanced for loop

Let me first show how the two code snippets given above can be simplified and then I will get into the details:

```
String[] values = { "a", "bb", "ccc" };
for(String value : values){
  System.out.println(value); //do something with each value
}
```

and

```
Set s = new HashSet();
s.add("a");
s.add("bb");
s.add("ccc");

for(Object value : s){
  System.out.println(value); //do something with each value
```

```
}
```

Isn't that simple? It reads easy too - "for **each** `value` in `values` do ..." and "for **each** `value` in `s` do ...".

8.5.2 Syntax of the enhanced for loop

The syntax of the enhanced for loop is as follows:

```
for( Type variableName : array_or_Iterable ){
   statement(s);
}
```

or if there is only one statement in the for block:

```
for( Type variableName : array_or_Iterable ) statement;
```

Type is the type of the elements that the array or the collection contains, **variableName** is the local variable that you can use inside the block to work with an element of the array or the collection, and **array_or_iterable** is the array or an object that implements `java.lang.Iterable` interface.

I know that I have been using the word "collection" up till now and suddenly I have switched to **Iterable**. The reason for the switch is that `Iterable` is a super interface of `java.util.Collection` and although for-each loop is used mostly to iterate over collections, technically, it can iterate through an object of any class that implements `Iterable`. In other words, any class that wants to allow a user to iterate through the elements that it contains using the enhanced for loop must implement the `Iterable` interface.

The `Iterable` interface was introduced in Java 1.5 specifically to denote that an object can be used as a target of the for-each loop. It has only one method named `iterate`, which returns a `java.util.Iterator` object. Since `java.util.Collection` extends `Iterable`, all standard collection classes such as `HashSet`, and `ArrayList` can be iterated over using the enhanced for loop.

For the purpose of the exam, you don't need to worry about the **Iterable** or the **Iterator** interface but it is a good idea to develop a mental picture of the relationship between the Iterable and the Iterator interfaces. Always remember that for-each can only be used to **"iterate"** over an object that is **"Iterable"**.

Note that `java.util.Iterator` itself is not a collection of elements and does not implement `Iterable`. Therefore, an Iterator object cannot be a target of a for-each loop. Thus, the following code will not compile:

```
Iterator it = myList.iterator();
for(Object s : it){  //this line will not compile
}
```

For the purpose of the JFCJA exam, you only need to know that you can use the for-each loop to iterate over any collection such as a **List** and an **ArrayList**.

8.5.3 Enhanced for loop in practice

Using enhanced for loop with typified collections

In the code that I showed earlier, I used Object as the type of the variable inside the loop. Since I was only printing the object out I didn't need to cast it to any other type. But if you want to invoke any type specific method on the variable, you would have to cast it explicitly like this:

```
List names = //get names from somewhere
for(Object obj : names){
  String name = (String) obj;
  System.out.println(name.toUpperCase());
}
```

The true power of the enhanced for loop is realized when you use generics (which were also introduced in Java 5, by the way) to generify the collection that you are trying to iterate through. Here is the same code with generics:

```
List< String>  names = //get names from somewhere
for(String name : names){
   System.out.println(name.toUpperCase());
}
```

Although the topic of generics is not on the JFCJA exam, you will see questions on foreach that use generics. You don't need to know much about generics to answer the questions, but you should be aware of the basic syntax. I will discuss it more while talking about ArrayList later.

Counting the number of iterations

In a regular for loop, the iteration variable (usually named i) tells you which iteration is currently going on. There is no such variable in a foreach loop. If you do want to know about the iteration number, you will need to create and manage another variable for it. For example:

```
List<String> names = //get names from somewhere
int i = 0;
for(String name : names){
   i++;
   System.out.println(i+" : "+name.toUpperCase());
}
System.out.println("Total number of names is "+i);
```

8.6 Use break and continue

8.6.1 Terminating a loop using break

A loop doesn't always have to run its full course. For example, if you are iterating through a list of names to find a particular name, there is no need to go through the rest of the iterations after you have found that name. The break statement does exactly that. Here is the code:

```
String[] names = { "ally", "bob", "charlie", "david" };
for(int i=0; i<names.length; i++){
  System.out.println(names[i]);
  if("bob".equals(names[i])){
     System.out.println("Found bob at "+i);
     break;
  }
}
```

The output of the above code is:

```
ally
bob
Found bob at 1
```

Observe that `charlie` and `david` were not printed because the loop was broken after reaching `bob`.

It doesn't matter which kind of loop (i.e. while, do-while, for, or enhanced for) it is. If the code in the loop encounters a break, the loop will be broken immediately. The condition section and the updation section (in case of a for loop) will not be executed anymore and the control will move to the next statement after the loop block.

8.6.2 Terminating an iteration of a loop using continue

The **continue** statement is a little less draconian than the **break** statement. The purpose of a **continue** statement is to just skip the rest of the statements in the loop while executing that particular iteration. In other words, when a loop encounters the continue statement, the remaining statements within the loop block are skipped. The rest of the steps of executing a loop are performed as usual, i.e., the control moves on to updation section (if it is a for loop) and then it checks the loop condition to decide whether to execute the next iteration or not.

This is useful when you don't want to execute the rest of the statements of a loop after encountering a particular condition. Here is an example:

```
String[] names = { "ally", "bob", "charlie", "david" };

for(String name : names){ //using a for-each loop this time

  if("bob".equals(name)){
     System.out.println("Ignoring bob!");
     continue;
  }
  System.out.println("Hi "+name+"!");
}
```

This code produces the following output:

```
Hi ally!
Ignoring bob!
```

```
Hi charlie!
Hi david!
```

Observe that the print statement saying `Hi` was skipped only when the name was `bob`.

Just like the `break` statement, the `continue` statement is applicable to all kind of loops.

Both of these statements are always used in conjunction with a conditional statement such as an if/if-else. If a continue or a break executes unconditionally then there would be no point in writing the code below a continue or a break. For example, the following code will fail to compile:

```
String[] names = { "ally", "bob", "charlie", "david" };

for(int i=0; i<names.length; i++){
    continue; //or break;
    System.out.println("Hi "+names[i]+"!"); //This line will never get to execute
}
```

The compiler will complain that the print statement is unreachable.

8.7 Nested loops

8.7.1 Nested loop

A nested loop is a loop that exists inside the body of another loop. This is not a big deal if you realize that the body of a loop is just another set of statements. Among these set of statements, there may also be a loop statement. For example, while looping through a list of Strings, you can also loop through the characters of each individual String as shown in the following code:

```
String[] names = { "ally", "bob", "charlie", "david" };

for(String name : names) {

    int sum = 0;

    for(int i=0; i<name.length(); i++){
        char ch = name.charAt(i);
        int letterNumber = ch - 96; //converts an 'a' to 1, 'b' to 2, etc.
        sum = sum + letterNumber;
    }

    System.out.println("Lucky number for "+name+" is "+sum);
}
```

outer loop

inner loop

Example of a nested loop

The above code nests a regular for loop inside an enhanced for loop. The following is the output produced by this code:

```
Lucky number for ally is 50
Lucky number for bob is 19
```

```
Lucky number for charlie is 56
Lucky number for david is 40
```

One thing that you need to be very careful about when using nested loops is managing the loop variables correctly. Consider the following program that is supposed to calculate the sum of all values in a given multidimensional array of ints:

```
int[][] values = { {1, 2, 3} , { 2, 3}, {2 },  { 4, 5, 6, 7 } };

int sum = 0;
for(int i = 0; i<values.length; i++) {
   for(int j=0; j<values[i].length; i++) {
        sum = sum + values[i][j];
   }
}
System.out.println("Sum is "+sum);
```

Read the above code carefully and try to determine the output.

It actually throws an exception: `Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4`

Observe that the inner for loop is incrementing `i` instead of incrementing `j`. Now, let's execute the code step by step:

Step 0: Before the outer loop starts, `values.length` is 4 and sum is 0.
Step 1: Outer loop is encountered - `i` is initialized to 0, `i<values.length` is `true` because 0 is less than 4, so, the loop is entered.
Step 2: Inner loop is encountered - `j` is initialized to 0, `j<values[i].length` is true because `i` is 0, therefore, `values[i]` refers to `{ 1, 2, 3}` and `values[i].length` is 3. Therefore, the loop is entered.
Step 3: Loop body is executed. `sum` is assigned `0 + values[0][0]`, i.e., `1`. `sum` is now 1.
Step 4: Inner loop body is finished. Inner loop's updation section is executed, so, `i` is incremented to 1.
Step 5: Inner loop's condition `j<values[i].length` is executed and it evaluates to `true` because `j` is still 0 and `i` is 1, so, `values[i]` refers to `{2, 3}` and `values[i].length` is 2. Thus, second iteration of the inner loop will start.
Step 6: Loop body is executed. `sum` is assigned `1 + values[1][0]`, i.e., 1+2. `sum` is now 3.
Step 7: Inner loop body is finished. Inner loop's updation section is executed, so, `i` is incremented to 2.
Step 8: Inner loop's condition `j<values[i].length` is executed and it evaluates to `true` because `j` is still 0 and `i` is 2, so, `values[i]` refers to `{2}` and `values[i].length` is 1. Thus, third iteration of the inner loop will start.
Step 9: Loop body is executed. `sum` is assigned `3 + values[2][0]`, i.e., 3+2. `sum` is now 5.
Step 10: Inner loop body is finished. Inner loop's updation section is executed, so `i` is incremented to 3.
Step 11: Inner loop's condition `j<values[i].length` is executed and evaluates to `true` because

`j` is still 0 and `i` is 3, so, `values[i]` refers to `{4, 5, 6, 7 }` and `values[i].length` is 4. Thus, fourth iteration of the inner loop will start.
Step 12: Loop body is executed. `sum` is assigned `5 + values[3][0]`, i.e., `5 + 4`. `sum` is now 9.
Step 13: Inner loop body is finished. Inner loop's updation section is executed, so, `i` is incremented to 4.
Step 14: Inner loop's condition `j<values[i].length` is executed. Now, here we have a problem. `i` is 4 and because the length of the `values` array is only 4, `values[4]` exceeds the bounds of the array and therefore `values[i]` throws an `ArrayIndexOutOfBoundsException`.

This is a very common mistake and while in this case the exception message made it easy to find, such mistakes can actually be very difficult to debug as they may produce plausible but incorrect results. That is why, although there is no technical limit to how many levels deep you can nest the loops, most professionals avoid nesting more than two levels.

Exam Tip

In the exam you will be presented with questions containing two levels of loops. Many students ask if there is an easy way to figure out the output of such loops. Unfortunately, there is no short cut. It is best if you execute the code step by step while keeping track of the variables at each step as shown above on a piece of paper. The code in the exam will be tricky and it may trip you up even if you are an experienced programmer.

As an exercise, try executing the same code given above after changing `i++` to `j++`.

8.7.2 breaking out of and continuing with nested loops

You can use break and continue statements in nested loops exactly like I showed you earlier with single loops. They will let you break off or continue with the loop in which they are present. For example, check out the following code that tries to find `"bob"` in multiple groups of names:

```
String[][] groups = { { "ally", "bob", "charlie" } , { "bob", "alice", "boone"}, {
   "chad", "dave", "elliot" }  };

  for(int i = 0; i<groups.length; i++){
     for(String name : groups[i]){
       System.out.println("Checking "+name);
       if("bob".equals(name)) {
          System.out.println("Found bob in Group "+i);
          break;
       }
     }
  }
```

The following is the output of the above code:

```
Checking ally
Checking bob
```

```
Found bob in Group 0
Checking bob
Found bob in Group 1
Checking chad
Checking dave
Checking elliot
```

When the inner loop encounters the String "bob", it executes break, which causes the inner loop to end immediately. The control goes on to execute the next iteration of the outer loop, i.e., it starts looking for "bob" in the next group of names. This is the reason why the above output does not contain "Checking charlie", "Checking alice", and "Checking boone".

Now what if you want to stop checking completely as soon as you find "bob" in any of the groups? In other words, what if you want to break out of not just the inner loop but also out of the outer loop as soon as you find "bob"? Java allows you to do that. You just have to specify which loop you want to break out of. Here is how:

```
String[][] groups = { { "ally", "bob", "charlie" } ,
                                { "bob", "alice", "boone"},
                                { "chad", "dave", "elliot" }  };

 MY_ OUTER_ LOOP: for(int i = 0; i<groups.length; i++){
     for(String name : groups[i]){
       System.out.println("Checking "+name);
       if("bob".equals(name)) {
          System.out.println("Found bob in Group "+i);
          break MY_ OUTER_ LOOP;
       }
     }
  }
```

The following is the output of the above code:

```
Checking ally
Checking bob
Found bob in Group 0
```

I just **"labeled"** the outer for loop with MY_OUTER_LOOP and used the same label as the target of the break statement. This is known as a **"labeled break"**. A **"labeled continue"** works similarly.

Let me give a quick overview of labels and then I will get back to the rules of using labeled break and continue.

What is a label?

A **label** is nothing but a name that you generally give to a **block of statements** or to statements that allow block of statements inside them (which means if, for, while, do-while, enhanced for, and

switch statements). The exam does not test you on the precise rules for applying labels or naming of labels, but here are few examples:

```
SINGLE_STMT: System.out.println("hello");

HELLO: if(a==b) callM1(); else callM2();

COME_HERE : while(i=>0) {
 System.out.println("hello");
}

SOME_LABEL:  {  //ok, because this is a block of statements
 System.out.println("hello1");
 System.out.println("hello2");
}
```

Here are some examples of invalid usage of labels:

```
BAD1 : int x = 0; //can't apply a label to declarations

BAD2 : public void m1() { } //can't apply a label to methods
```

Although not necessary, the convention is to use capital letters for label names. Also, applying a label to a statement doesn't necessarily mean that you have to use that label as a target of any break or continue. You can label a statement just for the sake of it :)

Rules for labeled break and continue

Getting back to using labeled break and continue, you need to remember that if you use a labeled break or a labeled continue, then that label must be present on one of the nesting loop statements within which the labeled break or continue exists. Thus, the following code will fail to compile:

```
LABEL_1 : for(String s :  array) System.out.println(s); //usage of LABEL_1 is valid here
for(int i = 0; i<10; i++){
  if(i ==2) continue LABEL_1; //usage of continue is invalid because LABEL_1 does not
    appear on a loop statement that contains this continue.
}
```

The following is invalid as well:

```
for(int i = 0; i<10; i++){
  LABEL_1 : if(i ==2) System.out.println(2); //usage of LABEL_1 is valid here
  for(int j = 0; j<10; j++){
     if(i ==2) continue LABEL_1; //usage of continue is invalid because LABEL_1 does
    not appear on a loop statement that contains this continue.
  }
}
```

But the following is valid because the continue statement uses a label that nests the continue statement.

```
LABEL_1 : for(int i = 0; i<10; i++){
  if(i ==2) System.out.println(2);
  for(int j = 0; j<10; j++){
     if(i ==2) continue LABEL_1; //usage of continue is valid because it refers to an
    outer loop
  }
}
```

You can use labeled break and continue for non-nested loops as well but since unlabeled break and continue are sufficient for breaking out of and continuing with non-nested loops, labels are seldom used in such cases.

It is actually possible to use a labelled break (but not a labelled continue) inside any block statement. The block doesn't necessarily have to be a loop block. For example, the following code is valid and prints 1 3 .

```
public static void main (String[] args) {
   MYLABEL: {
      System.out.print("1 ");
      if( args != null) break MYLABEL;
      System.out.print("2 ");
   }
   System.out.print("3 ");
}
```

However, you need not spend time in learning about weird constructs because the exam doesn't test you on obscure cases. The `break` and `continue` statements are almost always used inside loop blocks and that is what the exam focuses on.

To answer the questions on break and continue in the exam correctly, you need to practice executing simple code examples on a piece of paper. The following is one such code snippet:

```
public static void doIt(int h){
  int x = 1;
  LOOP1 : do{

    LOOP2 : for(int y = 1; y < h; y++ ) {

        if( y == x ) continue;

        if( x*x + y*y == h*h){
          System.out.println("Found "+x+" "+y);
          break LOOP1;
        }
      }
      x++;
  }while(x<h);
}
```

Execute the above code mentally or on a piece of paper and find out what will it print when executed with 5 as an argument and then 6 as an argument.

8.8 Comparing loop constructs

8.8.1 Comparison of loop constructs

As you saw before, there are only a few technical differences between the four kind of loops. The **for, foreach,** and **while** loops are conceptually a little different than the **do-while** loop due to the fact that a **do-while** loop always executes at least one iteration. The **foreach** loop is a little different than other loops due to the fact that it can be used only for arrays and for classes that implement `java.lang.Iterable` interface.

Other than these differences, they are all interchangeable. For example, you can always replace a **while** loop with a **for** loop as follows:

```
while( booleanExpression ){

}

for( ; booleanExpression ; ){

}
```

You can also replace a **foreach** loop with a **for** loop as shown below:

```
for( Object obj :  someIterable){

}


for( Iterator it = someIterable.iterator() ; it.hasNext() ; ){
  Object obj = it.next();
}
```

Of course, the foreach version is a lot simpler than the for version and that is the point. While you can use any of the loops for a given problem, you should use the one that results in code that is most understandable or intuitive.

A general rule of thumb is to use a for loop when you know the number of iterations before hand and use a while loop when the decision to perform the next iteration depends on the result of the operations performed in the prior iteration. For example, if you want to print "hello" ten times, use a for loop, but if you want to keep printing "hello" until the user enters a secret code as a response, use a while loop!

8.9 Exercise

Note: Since most of the questions on loops involve arrays, do the following questions after going through the chapter on Arrays.

1. Initialize an array of 10 ints using a for loop such the each element contains an value equal to the sum of its previous two values. Do the same using a while loop.

2. Write a method that determines whether a given number N is a prime number or not by dividing that number with all the numbers from 2 to N/2 and checking the remainder.

3. Use nested for loops to print a list of prime numbers from 2 to N.

4. The following code contains a mistake. Identify the problem, fix it, and print out all the elements of the chars array.

```
String[] chars = new String[4];
char cha = 97;
for(char c=1;c<=chars.length; c++){
        chars[c] = ""+cha;
        cha++;
    }
}
```

5. To avoid the possibility of inadvertently introducing the mistake shown in the above code, a programmer decided to use for-each loops instead of the regular for loops:

```
String[] chars = new String[4];
char cha = 97;
for(String s : chars){
        s = ""+cha;
        cha++;
}
```

   Will this work? Why/why not?

6. Use an enhanced for loop to print alternate elements of an array. Can you use an enhanced for loop to print the elements in reverse order?

7. Given two arrays of same length and type, copy the elements of the first array into another in reverse order.